



# CompSci 130

## Introduction to Software Fundamentals

Recursion

# Agenda & Reading

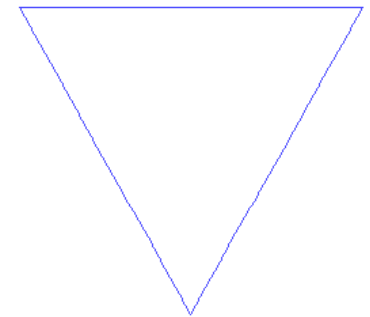
## ► Topics:

- What is recursion?
- Writing recursive code
- Tracing recursive code
- Examples (Binary Search & Fibonacci)



## ► Further Study:

- Online Textbook:
  - <https://onlinetextbook.auckland.ac.nz/6-recursion/>
- Problem Solving with Algorithms & Data Structures
- Chapter 5: Recursion



*The Koch snowflake,  
an example for a  
"fractal"*



# Recursion

"To understand recursion, you have to understand recursion!"

- Informally, recursion is when **something** is **defined in terms of itself**

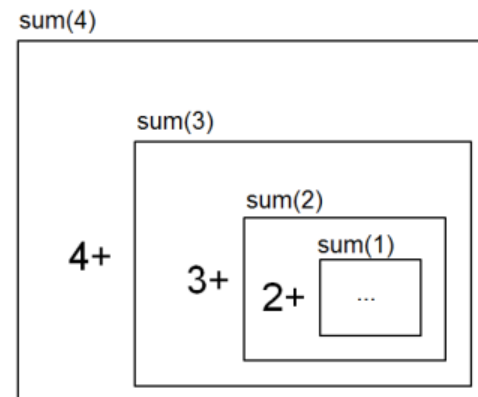


- Example: A familiar mathematical formula

$$\sum_{i=1}^n i = 1 + 2 + 3 + \dots + n$$

$$\text{sum}(n) = n(n+1) / 2 \quad \left\{ \begin{array}{l} \text{non-recursive} \\ \text{definition} \end{array} \right.$$

$$\text{sum}(n) = n + \text{sum}(n-1) \quad \left\{ \begin{array}{l} \text{recursive} \\ \text{definition} \\ \text{refers to itself} \end{array} \right.$$





# Recursion

$$\sum_{i=1}^n i = 1 + 2 + 3 + \dots + n$$

- ▶ In computer science, recursion usually refers to solving a problem by **splitting** it into **smaller problems** through defining a function that **calls itself as part of its own definition!**
- ▶ To solve a problem recursively:
  - ▶ Break the problem down into a smaller version of itself (usually by removing some piece which can be trivially combined with the smaller version's solution).
  - ▶ Then assume the smaller version can be solved somehow “automagically”...
  - ▶ Finally, combine the trivial part with the smaller problem to solve the original problem.

For example, we would like to find the sum of the first 6 numbers:

assume that we can “somehow” automatically solve the smaller problem  
i.e. **sum = 15**

**sum = 15**

**sum = 21**

break the problem into a smaller version:  
i.e. **5 numbers + 1 number**

combine the trivial part with the solution of the smaller version of the problem to solve the original problem



## Example 2

- ▶ Our **recursive** solution to this problem will **not** use a **loop**!
  - ▶ Instead, our function definition will include a **call to itself**
    - ▶ We will see later that this effectively performs a similar operation to the loop.

$$total = (1 + (3 + (5 + (7 + 9))))$$

$$total = (1 + (3 + (5 + 16)))$$

$$total = (1 + (3 + 21))$$

$$total = (1 + 24)$$

$$total = 25$$

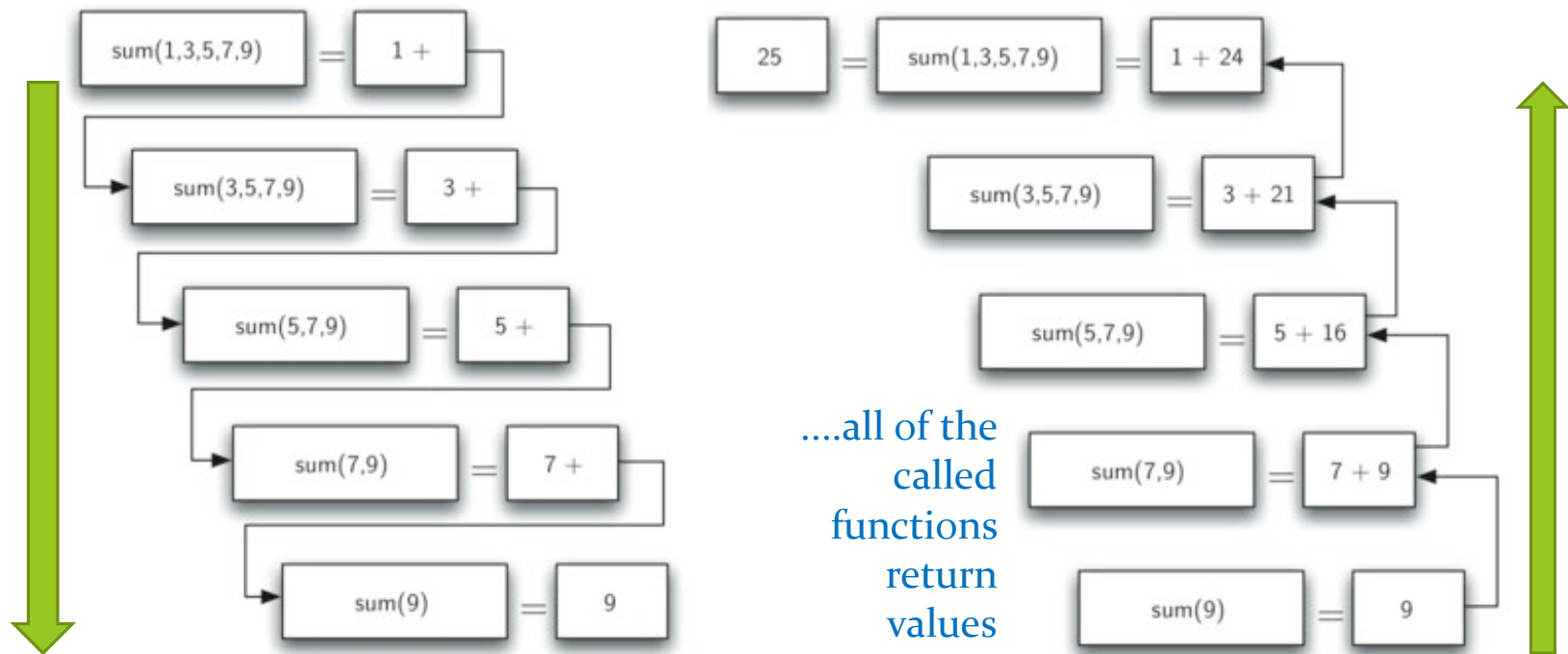
- ▶ Consider this list: **[1, 3, 5, 7, 9]**
- ▶ Recursive computation:

$$\text{sum}(\text{my\_list}) = \overbrace{\text{my\_list}[0]}^{\text{trivial case}} + \overbrace{\text{sum}(\text{my\_list}[1:])}^{\text{smaller sub-problem}}$$

$$\begin{array}{ccccc} \text{the sum of all} & & & & \\ \text{values in a} & & & & \\ \text{Python list} & = & \text{the first value in} & + & \text{the sum of the} \\ & & \text{the list} & & \text{rest of the list} \end{array}$$

# Recursion

- But, how does this actually work?



**base case**

A series of  
function calls  
occur....



... then ...



# Recursion

- ▶ This leads to the following recursive function for calculating the sum of the values in a Python list:

```
def sum_list(my_list):  
    if len(my_list) == 1:  
        return my_list[0]  
    else:  
        return my_list[0] + sum_list(my_list[1:],
```

this check is called the  
“base case” of the  
recursion, and is critical

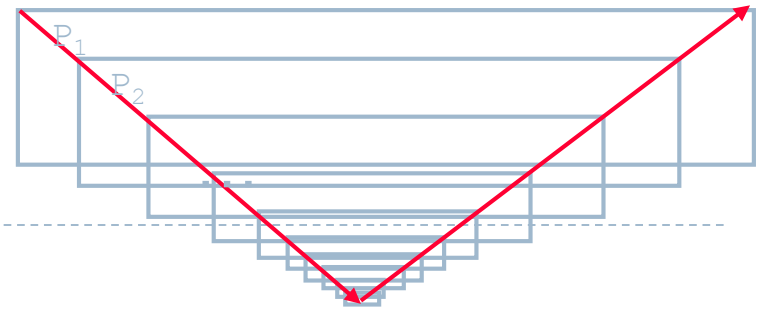
- ▶ Note:

- ▶ Complex problems can have simple recursive solutions
- ▶ Recursion is an alternative to iterative (loop-based) solutions
- ▶ BUT: Some recursive solutions are inefficient and impractical

non-recursive solution  
i.e. using a loop

```
def sum_list(my_list):  
    result = 0  
    for number in my_list:  
        result += number  
    return result
```

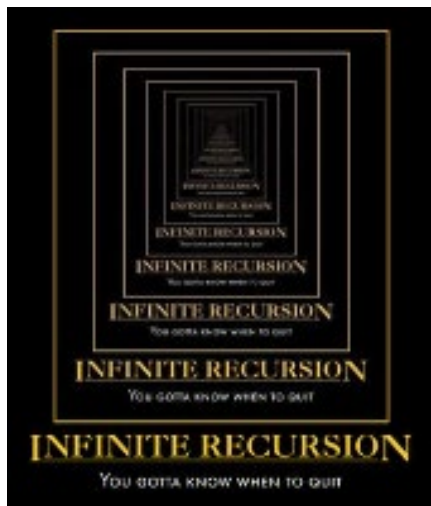
# Recursive Solutions



## ► Properties of a recursive solution:

1. A recursive function **calls itself**
2. Each recursive call solves an **identical**, but **smaller** problem
3. A **test** for the **base case** enables the recursive calls to stop (avoid infinite calls!)
  - Base case: a known, trivial case in a recursive definition
4. Eventually, one of the smaller problems must be the base case. The problem constantly becomes smaller but is not allowed to become smaller than the base case.

A **base case** is a special case whose solution is trivial





# Example: The factorial of n

- ▶ Compute the factorial of an integer  $n \geq 0$
- ▶ N-factorial can be defined recursively or non-recursively

- ▶ Non-recursive solution: an iterative definition of factorial(n)

- ▶ If  $n = 0$ ,  $\text{factorial}(0) = 1$

$$4! = 4 * 3 * 2 * 1 = 24$$

- ▶ If  $n > 0$ ,  $\text{factorial}(n) = n * (n-1) * (n-2) * \dots * 1$

```
def factorial(n):  
    result = 1  
    for i in range(n, 1, -1):  
        result = result * i  
    return result
```

- ▶ Recursive solution

- ▶ Recursive solution uses solution to a smaller sub-problem i.e.  $\text{factorial}(n-1)$
  - ▶ This leads to a recurrence relation (mathematical formula):

$$\text{factorial}(n) = n * [(n-1) * (n-2) * \dots * 1] = n * \text{factorial}(n-1)$$

- ▶ 
$$\text{factorail}(n) = \begin{cases} 1 & \text{if } n = 0 \\ n * \text{factorail}(n-1) & \text{if } n > 0 \end{cases}$$

```
def factorial(n):  
    if n <= 0:  
        return 1  
    else:  
        return n * factorial(n-1)
```

Recursive solution where  
function calls *itself*



# Four properties of a recursive solution

---

► factorial(n) satisfies our 4 criteria:

1. factorial **calls itself**
2. At each recursive call, the integer for which the factorial is computed, is **lowered by one**. So we solve smaller and smaller identical sub-problems.
3. The function handles the factorial for  $n=0$  differently from other factorials ( $\text{factorial}(0) = 1$ ). Thus, the **base case occurs when  $n=0$** .
4. Given that  $n$  is non-negative, the second property assures that the computation will always reach the base case.



# How can we debug recursive functions?

- ▶ We can use a call tree to **trace** a recursive function call
  - ▶ A systematic way to trace the actions of a recursive function
    - ▶ Create a new **box** for each recursive function call
    - ▶ Describe how return value is computed
    - ▶ Provide **link to box** (or boxes) for recursive function calls within the current function call
    - ▶ Each box corresponds to an activation record
      - Contains function's local environment at the time of and as a result of the call to the function

**factorial(3)**

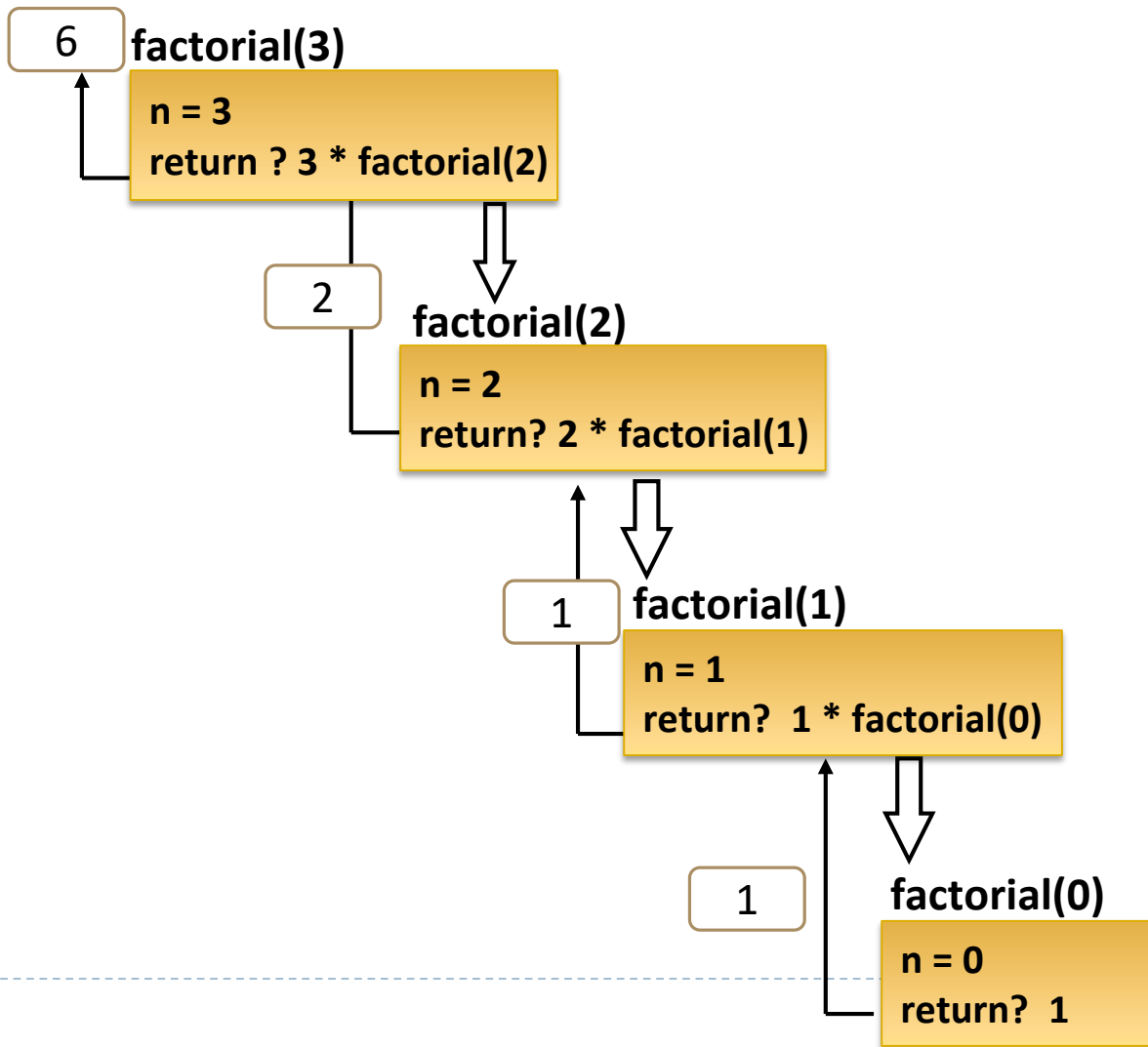
**n = 3**  
**return 3 \* factorial(2)**



```
def factorial (n) :  
    if n <= 0:  
        return 1  
    else:  
        return n * factorial (n-1)
```

Need to evaluate this  
before we can return

# Tracing recursive code for factorial



```
def factorial(n):  
    if n <= 0:  
        return 1  
    else:  
        return n * factorial(n)
```

## Exercise 1-3

```
def bad_sum(n):  
    return n + bad_sum(n-1)
```

► Q1: Consider the function:

- What will happen when you call the above function by `"print(bad_sum(2))"`?

► Q2: Consider the function:

```
def bad_sum(n):  
    if n == 0:  
        return 0  
    return n + bad_sum2(n-2)
```

- What will happen when you call the above function by `"print(bad_sum(2))"` and `print(bad_sum(3))`?

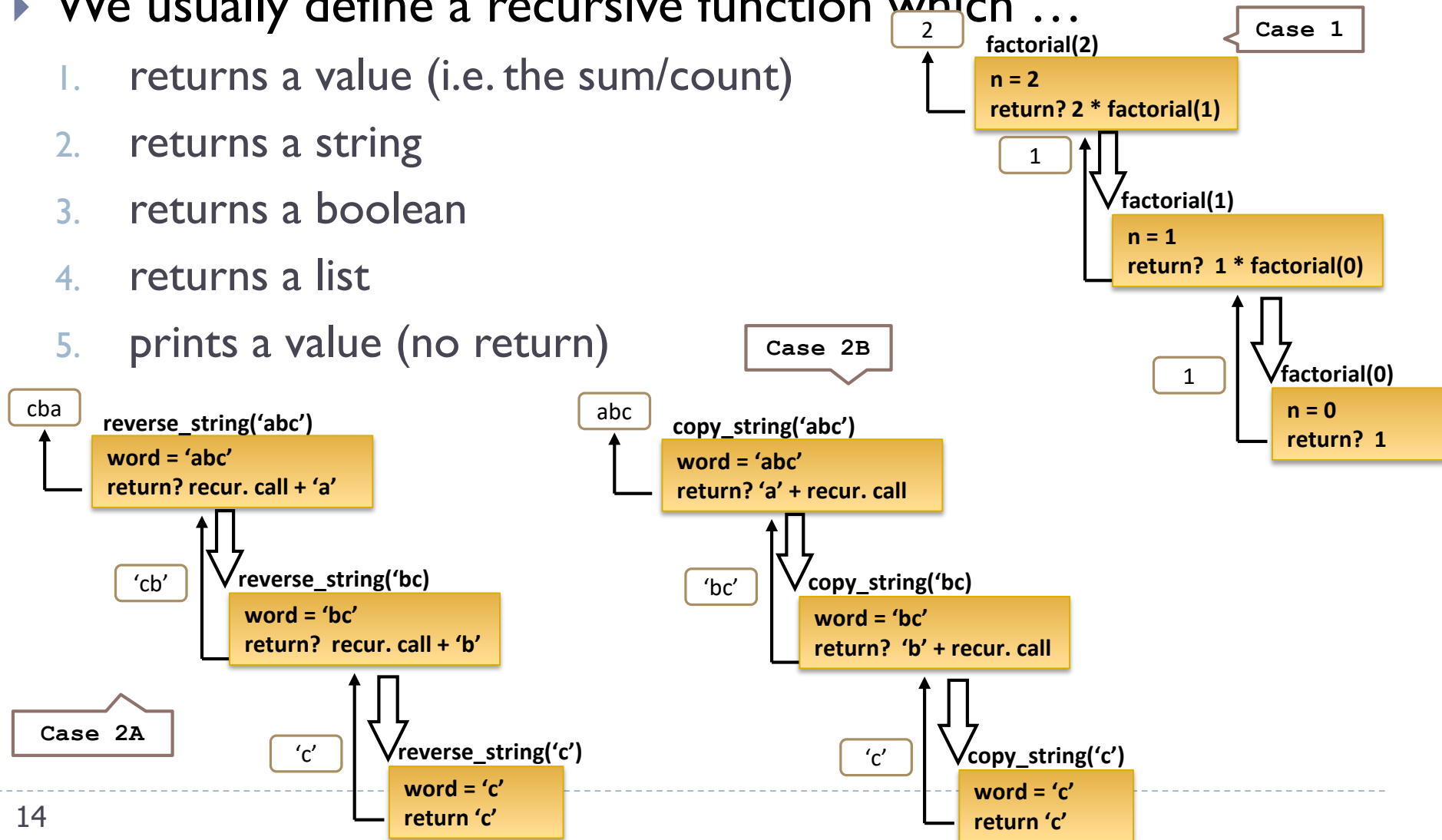
► Q3: `get_sum(n)`

- calculates the sum from 1 to n (i.e.  $1 + 2 + \dots + n$ ) **recursively** for integer  $n \geq 1$ .
  - `get_sum(5)` =  $5 + 4 + 3 + 2 + 1 = 15$
  - `get_sum(6)` =  $6 + 5 + 4 + 3 + 2 + 1 = 21$

## Exercise 4-6

► We usually define a recursive function which ...

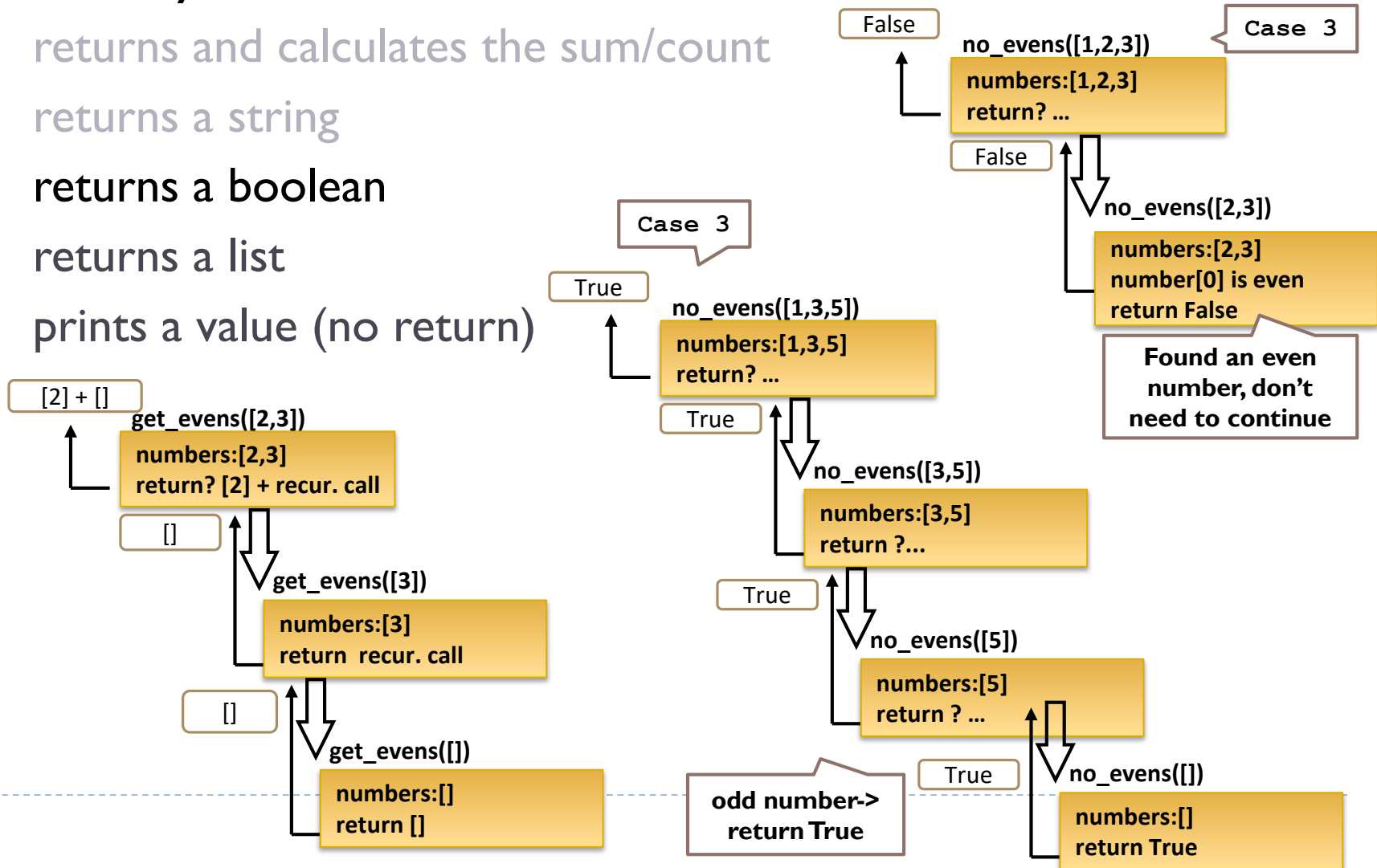
1. returns a value (i.e. the sum/count)
2. returns a string
3. returns a boolean
4. returns a list
5. prints a value (no return)



# Exercise 4-6

► We usually define a recursive function which ...

1. returns and calculates the sum/count
2. returns a string
3. returns a boolean
4. returns a list
5. prints a value (no return)



# Exercise 7

- ▶ `number_of_digits(n)`
  - ▶ returns the number of digits of the parameter integer
    - ▶ We can determine how many digits a positive integer has by repeatedly dividing by 10 (without keeping the remainder) until the number is less than 10, consisting of only 1 digit. We add 1 to this value for each time we divided by 10. Here is the recursive algorithm:
      - If  $n < 10$  return 1
      - Otherwise, return  $1 + \text{the number of digits in } n/10$  (ignoring the fractional part).
  - ▶ Implement this recursive algorithm in Python and test it using the values 15, 105, 1050, and 15105.

```
print(num_digits(15))  
print(num_digits(105))  
print(num_digits(1050))  
print(num_digits(15105))
```

```
2  
3  
4  
5
```

# Recursion and Binary Search



## ► Problem

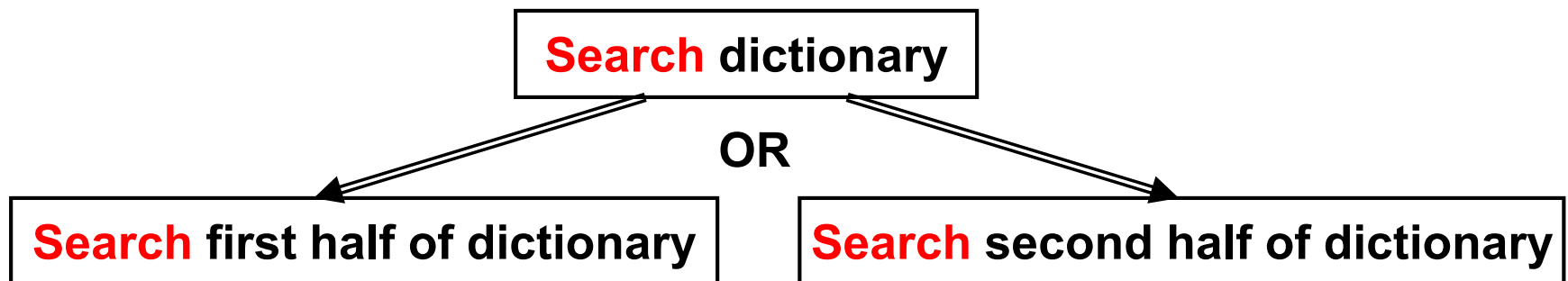
- Look for an element (key) in an ordered collection

## ► Sequential search

- Looks at every item in the collection in order until item is found:  $O(n)$

## ► Binary search

- Repeatedly halves the collection and determines which half contains the item.
- This sounds like a scenario where we could use recursion!





# Recursion and Binary Search

---

- ▶ Binary search recursive implementation
  - ▶ How to pass “half of the list” to the recursive calls of `binary_search`?
  - ▶ How to determine which “half of the list” contains the value?
  - ▶ What is (are) the base case(s)?
  - ▶ How will `binary_search` indicate the result of the search (value found or not)?
- ▶ Example: a **sorted list** of size 10

|   |   |   |   |    |    |    |    |    |    |
|---|---|---|---|----|----|----|----|----|----|
| 1 | 3 | 6 | 7 | 11 | 13 | 14 | 18 | 22 | 25 |
|---|---|---|---|----|----|----|----|----|----|

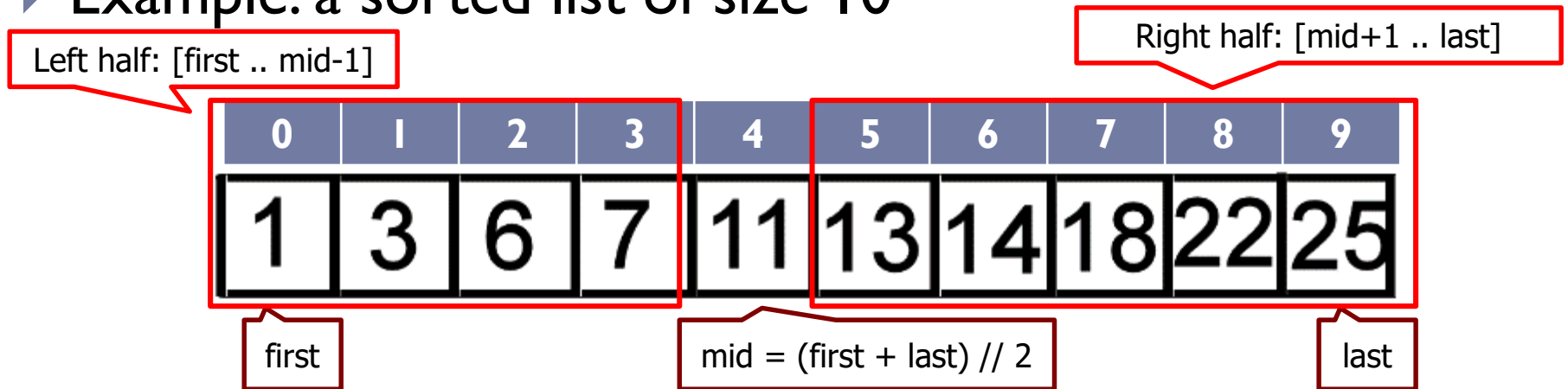


# Recursion and Binary Search

## ▶ Binary search recursive implementation

- ▶ How to pass “half of the list” to the recursive calls of `binary_search`?
- ▶ How to determine which “half of the list” contains the value?
- ▶ What is (are) the base case(s)?
- ▶ How will `binary_search` indicate the result of the search (value found or not)?

## ▶ Example: a sorted list of size 10



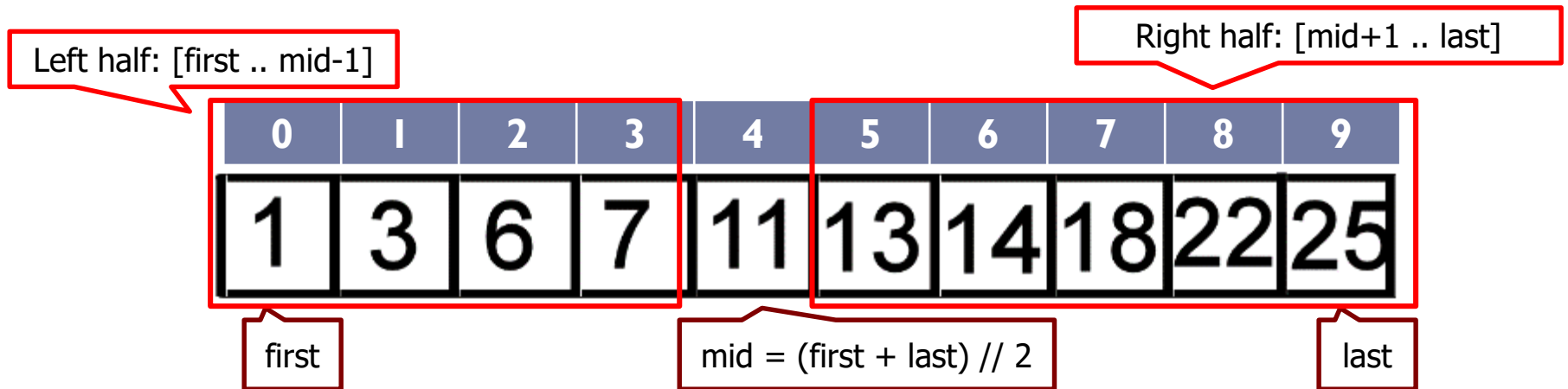
# Recursion and Binary Search

## ► Base case:

- If the list is empty (i.e. the number is not in the list) then return -1, OR
- If the middle element of the list is the one we are looking for, return its index

## ► Recursive call:

- Determine the element in the middle
  - If value we are searching for is smaller than middle element, search **left** half
  - If value we are searching for is larger than middle element, search **right** half





# Binary search: Call tree and box trace

index

| 0 | 1 | 2 | 3  | 4  | 5  | 6  | 7  |
|---|---|---|----|----|----|----|----|
| 1 | 5 | 9 | 12 | 15 | 21 | 29 | 31 |

- ▶ A successful search for value 9:

value = 9

first = 0

last = 7

$\text{mid} = \frac{0+7}{2} = 3$

value < num\_list[3]

**binary\_search(num\_list, 0, 7, 9)**

value = 9

first = 2

last = 2

$\text{mid} = \frac{2+2}{2} = 2$

value == num\_list[2]!

**binary\_search(num\_list, 0, 2, 9)**

value = 9

first = 0

last = 2

$\text{mid} = \frac{0+2}{2} = 1$

value > num\_list[1]

**binary\_search(num\_list, 2, 2, 9)**

return index 2

# Binary search: Call tree and box trace

- An unsuccessful search for value **6**:

| 0 | 1 | 2 | 3  | 4  | 5  | 6  | 7  |
|---|---|---|----|----|----|----|----|
| 1 | 5 | 9 | 12 | 15 | 21 | 29 | 31 |

value = 6  
first = 0  
last = 7  
 $mid = \frac{0+7}{2} = 3$   
value < num\_list[3]

**binary\_search(num\_list, 0, 7, 6)**

value = 6  
first = 2  
last = 2  
 $mid = \frac{2+2}{2} = 2$   
value < num\_list[2]

**binary\_search(num\_list, 0, 2, 6)**

**binary\_search(num\_list, 2, 2, 6)**

value = 6  
first = 0  
last = 2  
 $mid = \frac{0+2}{2} = 1$   
value > num\_list[1]

**binary\_search(num\_list, 2, 1, 6)**

value = 6  
first = 2  
last = 1  
first > last  
return -1

return -1



# Binary search

---

## ► Complete the implementation

```
def binary_search(num_list, first, last, value):  
    index = 0  
    if first > last:  
        index = -1 # value not found!  
    else:  
        mid = (first + last) // 2  
        if value == num_list[mid]:  
            index = mid  
        elif value < num_list[mid]:  
            index = binary_search(  
                )  
        else:  
            index = binary_search(  
                )  
    return index
```



# Fibonacci numbers

## ► Problem

- The sequence of numbers where each number is the sum of the previous two numbers. For example, for  $n = 9$ : [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

## ► The recursive definition

$$F(n) = \begin{cases} n & \text{if } n = 0 \text{ or } n = 1; \\ F(n-1) + F(n-2) & \text{if } n \geq 2. \end{cases}$$

## ► Straight-forward recursive code

```
def fibonacci(n):  
    if n==0 or n==1:  
        return n  
    else:  
        return fibonacci(n-1) + fibonacci(n-2)
```



# Fibonacci numbers

## ► Problem

- The sequence of numbers where each number is the sum of the previous two numbers. For example, for  $n = 9$ : [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

## ► But: which implementation would you choose in practice?

## ► Iterative!

### ► Why?

```
def fibonacci(n):  
    if n==0 or n==1:  
        return n  
    else:  
        return fibonacci(n-1) + fibonacci(n-2)
```

```
def fibonacci(n):  
    if n==0 or n==1:  
        return n  
    prev = 1  
    prev_prev = 0  
    for i in range(2, n+1):  
        temp = prev + prev_prev  
        prev_prev = prev  
        prev = temp  
    return prev
```



# Fibonacci numbers

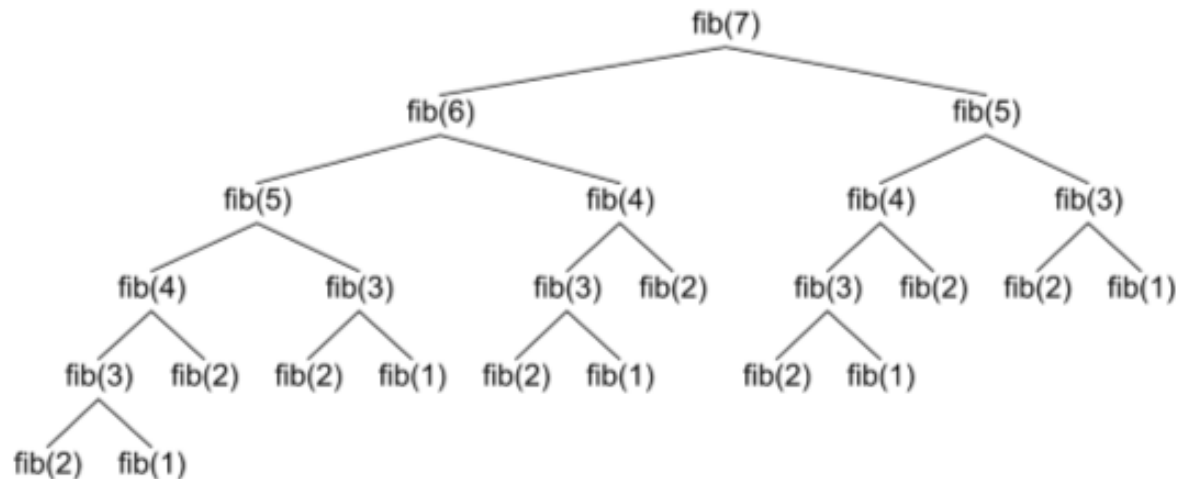
- ▶ Recursive vs iterative implementation
- ▶ Experiment:
  - ▶ How long does it take for fibonacci(100) to compute?

| Fibonacci number | Time taken for fibonacci(n) |                |
|------------------|-----------------------------|----------------|
|                  | recursive                   | iterative      |
| 10               | < 0.001 second              | < 0.001 second |
| 20               | < 0.001 second              | < 0.001 second |
| 30               | 1 second                    | < 0.001 second |
| 100              | ???                         | < 0.001 second |



# Fibonacci numbers

- ▶ What happens in recursive solution?
  - ▶ Let's have a closer look at the function calls...
  - ▶ “Memoization” is a technique to overcome this problem (not covered in this course)





# Fibonacci numbers

- ▶ Recursive implementation has exponential runtime  $O(2^n)$ !
- ▶ Actually  $O(1.618\dots^n)$ : Golden section, Golden ratio

| <b>n</b> |    | <b>n</b> <b>Fibonacci sequence up to</b> |                             |
|----------|----|------------------------------------------|-----------------------------|
| 2        | 1  | 20                                       | 6,765                       |
| 3        | 2  | 30                                       | 832,040                     |
| 4        | 3  | 40                                       | 102,334,155                 |
| 5        | 5  | 50                                       | 12,586,269,025              |
| 6        | 8  | 60                                       | 1,548,008,755,920           |
| 7        | 13 | 70                                       | 190,392,490,709,135         |
| 8        | 21 | 80                                       | 23,416,728,348,467,685      |
| 9        | 34 | 90                                       | 2,880,067,194,370,816,120   |
| 10       | 55 | 100                                      | 354,224,848,179,261,915,075 |



# Summary

---

- ▶ A recursive algorithm passes the buck repeatedly to the same function
- ▶ Recursive algorithms are well-suited for solving problems in domains that exhibit recursive patterns (trees, fractals)
- ▶ Recursive strategies can be used to simplify complex solutions to difficult problems
- ▶ Four questions for constructing recursive solutions
  - ▶ How can you define the problem in terms of a smaller problem of the same type?
  - ▶ How does each recursive call diminish the size of the problem?
  - ▶ What instance of the problem can serve as the base case?
  - ▶ As the problem size diminishes, will you reach this base case?